Software Engineering

Typescript Features and Paradigms 1

What we have learned...

- Introduction to Typescript
- Difference between Typescript and Javascript
- Understanding Typescript Nature and Syntax

What we will learn...

- Overview of Typescript features and paradigms
- TypeScript Data Types
- Type Inference and Annotation
- Functions and Function Signatures
- Advanced Type System
- Async Programming with TypeScript
- Interface
- Intersection and Type

Get to know our topic!

Let’s review
A bit !

TypeScript Features and Paradigms

TypeScript is a statically-typed superset of JavaScript designed to enhance code quality and maintainability. It brings features like static typing, interfaces, and modern JavaScript support.

TypeScript Review

```typescript
// TypeScript code
function greet(name: string): string {
  return `Hello, ${name}`;
}

const result = greet("John");
console.log(result);
```

TypeScript Review

One of the main features of TypeScript is **static typing**. This means you can define the types of variables, function parameters, and return values.

Static Typing: For example, you can **specify that a variable** is a string or a number, and TypeScript will check for type errors at compile time, helping to catch mistakes early.

How does TypeScript help find errors early?

Think of TypeScript as a **safety net**.

It's like having a guide that helps you **avoid common pitfalls** and ensures your **code is robust**.

Static typing allows catching errors at **compile-time**, providing a more stable and maintainable codebase.

TypeScript usage

Write a TypeScript function named `calculateSum` that takes two numbers and returns their sum.

```
function calculateSum(a: number, b: number): number {
  return a + b;
}
```

TypeScript's static typing allows for detecting potential errors during development, reducing the chances of runtime issues. If a user provides `calculateSum` with a type other than number, it will result in an error.

TypeScript Data Types

TypeScript supports various data types such as number, string, boolean, array, and custom types. These types provide clarity and structure to variables.

```
// TypeScript data types
let age: number = 25;
let name: string = "Alice";
let isStudent: boolean = true;
let hobbies: string[] = ["reading", "coding", "traveling"];
let person: { name: string, age: number } = { name: "Bob", age: 30 };
```

TypeScript Data Types

Think of data types as **containers** with **labels**. Each container holds a specific type of information, making it easier to organize and understand.

Why using Data Types?

Using data types prevents **unexpected values** and **enhances code** readability and maintainability.

TypeScript Data Types

How does using data types help in preventing potential errors in your code?

Data types **limit variables** to specific types, helping to **prevent mistakes** and **catch errors early**.

Declare a TypeScript variable named **studentCount** and assign it a number representing the total number of students in a class.

```
let studentCount: number = 30;
```

Type Inference and Annotation

TypeScript has type inference, which automatically determines types based on values, and type annotation, where developers explicitly declare types.

```
// Type inference
let message = "Hello, TypeScript!"; // TypeScript infers the type as string

// Type annotation
let count: number = 10; // Explicitly declaring the type as number
```

Type Inference and Annotation

Type inference is like a helpful assistant understanding your needs without explicit instructions, while type annotation is like providing clear guidelines.

Why we need Type inference?

Type inference allows flexibility without sacrificing type safety, and type annotation offers clear communication to developers.

Type Inference and Annotation

In what situations might you prefer using type annotation over relying on type inference?

Type annotation is useful when you want to provide **explicit guidance** or when type inference might **not capture** the intended type accurately.

Write a TypeScript function named calculateArea that takes the radius (a number) and returns the area of a circle (a number). Use type **annotation**.

```
function calculateArea(radius: number): number {
  return Math.PI * radius ** 2;
}
```

Functions and Function Signatures

Functions in TypeScript can have defined **parameter** and **return** types, providing clarity and structure to function declarations.

Here's a breakdown of a function signature in TypeScript:

1. **Function Name**: The name of the function.
2. **Parameters**: A list of input parameters, each with a type annotation.
3. **Return Type**: The type of value the function returns.

Functions and Function Signatures

Functions are like machines. Function signatures define the inputs and outputs, making it clear how to use them.

Function signatures improve code readability, help catch errors, and provide clear documentation.

```
// Function with explicit types
function add(x: number, y: number): number {
  return x + y;
}
```

The type of the value the function returns is specified after the parameters. In function add(a: number, b: number): number, : number indicates that the function returns a number.

Functions and Function Signatures

Here's an example with different parameter and return types:

```
function formatDate(date: Date): string {
    return date.toISOString();
}
```

The formatDate function takes a Date object as a parameter and returns a string.

Async Programming with TypeScript

TypeScript simplifies asynchronous programming through features like async/await, making code more readable and maintainable.

```typescript
// Async function
async function fetchData(): Promise<string> {
  const data = await fetchDataFromAPI();
  return data;
}
```

Async Programming with TypeScript

Async programming is like waiting for a bus. You don't block the **road**; instead, you continue with other activities until the bus arrives.

Async/await syntax in TypeScript simplifies asynchronous code, making it more **readable** and resembling **synchronous code**.

Async Programming with TypeScript

How does async/await improve the readability of asynchronous code in TypeScript?

Async/await allows developers to write asynchronous code in a synchronous style, making it easier to understand and reason about.

Write a TypeScript async function named fetchUserData that simulates fetching user data from an API with a delay of 2 seconds.

```typescript
async function fetchUserData(): Promise<User> {
  const data = await simulateFetchFromAPI<User>({ name: "John", age: 30 });
  return data;
}
```

Advanced Type System

TypeScript's advanced type system includes features like **intersection types, and interface** offering flexibility and reusability.

Advanced Type System

Primitive data type	advanced data type
Can you build this?	Can you build this?

https://i.pinimg.com/originals/ea/33/04/ea33048735f94e0b799f21831cdb161b.jpg

Advanced Type System

Advanced types are like building blocks. They allow developers to create more versatile and reusable code structures.

Advanced types enable developers to express complex relationships between types, enhancing code flexibility and maintainability.

```
// It's duplicate
const john: Person = {
  name: "John",
  age: 28,
};

const clara: Person = {
  name: "Clara",
  age: 30,
};
```

Interface

Interfaces in TypeScript allow you to define the structure of objects, providing a blueprint for the shape that an object should have. They can include properties, methods, and also extend other interfaces.

```typescript
// It's duplicate
const john: Person = {
  name: "John",
  age: 28,
};

const clara: Person = {
  name: "Clara",
  age: 30,
};
```

Interface

```
// Example of an interface for a User
interface User {
  id: number;
  username: string;
  email?: string; // Optional property
}

// Function accepting a User object
function printUser(user: User): void {
  console.log(`User ID: ${user.id}, Username: ${user.username}`);
  if (user.email) {
    console.log(`Email: ${user.email}`);
  }
}
```

Interface

Think of an interface like a **contract**.

Imagine you are designing a remote control. The remote control has buttons (properties) like volume up, volume down, and power. Each button has a specific purpose, and anyone designing a device (object) that uses this remote control must adhere to this contract.

Interfaces enforce a **structure in your code**, promoting **consistency** and helping **catch errors** early in the development process.

They also facilitate code collaboration by clearly defining the expectations for object shapes.

Interface

Create an interface named Book with properties title (string), author (string), and publishedYear (number). Use this interface in a function that accepts a Book object and prints its details.

```typescript
interface Book {
  title: string;
  author: string;
  publishedYear: number;
}

function printBookDetails(book: Book): void {
  console.log(`
    Title: ${book.title},
    Author: ${book.author},
    Published Year: ${book.publishedYear}
  `);
}

// Usage
const myBook: Book = {
  title: 'Example Book',
  author: 'John Doe',
  publishedYear: 2022
};

printBookDetails(myBook);
```

Write an interface of e-wallet, then declare a variable of e-wallet data with the data type of the interface.

Field : name as string, code as string, userId as string, balance as number

Write an interface of e-wallet, then declare a variable of e-wallet data with the data type of the interface.

Field : name as string, code as string, userId as string, balance as number

Wrap those into a function that accept e-wallet interface, console.log the e-wallet

Union Type

Union types allow a variable to hold values of multiple specified types, providing flexibility in data representation.

```
let identifier: number | string;
identifier = 123;
identifier = "ABC";
```

Intersection type

In TypeScript, an intersection type is a way to combine multiple types into one. It allows you to create a new type that has all the properties and methods of the types you're intersecting. You use the & operator to perform an intersection between types.

```typescript
interface Person {
  name: string;
  age: number;
}

interface Employee {
  employeeId: number;
  role: string;
}

type EmployeePerson = Person & Employee;

const employeePerson: EmployeePerson = {
  name: "John",
  age: 25,
  employeeId: 123,
  role: "Developer"
};
```

Make an intersection (type) called AcceptedPayment that that accept E-wallet or Bank.

Clue :
Write a new interface called Bank with field accountName as string, accountNumber as string, balance as number

Then merge the intersection of E-wallet and Bank.

Make an intersection (type) called AcceptedPayment that that accept E-wallet or Bank.

Clue :
Write a new interface called Bank with field accountName as string, accountNumber as string, balance as number

Then merge the intersection of E-wallet and Bank.

Make a function to deduct the money balance with a defined transaction amount with the acceptable payment method. Console.log the result

transact(balance, amount, paymentMethod (type))

QnA & Discussion

Today’s Recap

External Resources

- 📖 **Overview of TypeScript Features and Paradigms:**
  - TypeScript Documentation
  - What is TypeScript and why would you use it?
- 📖 **TypeScript Data Types:**
  - TypeScript Basic Types
  - TypeScript - Data Types
- 📖 **Type Inference and Annotation:**
  - TypeScript Type Inference
  - An Introduction to TypeScript: Static Typing
- 📖 **Functions and Function Signatures:**
  - TypeScript Functions
  - TypeScript Functions and Function Expressions
- 📖 **Advanced Type System:**
  - TypeScript Advanced Types
  - Advanced TypeScript Types with Examples
- 📖 **Async Programming with TypeScript:**
  - TypeScript Async Functions
  - Mastering Async Await in TypeScript
- 📖 **TypeScript Configuration and Compilation:**
  - TypeScript tsconfig.json
  - Understanding TypeScript Configuration

Thank you!

Rules are made to be understand,
Not to be followed.

Share your learning journey with us!
Let's inspire each other with your stories

@revou_id RevoU revoudotco